Microsoft Fabric Warehouse

Compute & SQL Engine

DBA Technical Reference | Volume 2 of 9

April 2025 | Version 1.0

Table of Contents

1. Fabric Compute Architecture

Fabric Warehouse uses a distributed, cloud-native SQL engine that decouples compute from storage. The compute layer reads Parquet files directly from OneLake and processes queries using a massively parallel processing (MPP) architecture.

Core compute architecture principles:

- Compute and storage are fully separated — compute can scale independently of data volume
- No persistent compute nodes — queries spin up compute on demand and release it when idle
- Distributed query processing across multiple nodes for large analytical workloads
- Intelligent query caching reduces repeated I/O against OneLake storage
- Automatic query plan generation — no manual hints or plan guides required

2. Capacity Units (CUs) Deep Dive

Every Fabric operation — query execution, data ingestion, pipeline runs — consumes Capacity Units. CUs represent a normalized measure of compute resources combining CPU, memory, and I/O.

Operation Type	CU Consumption Pattern
T-SQL SELECT queries	CUs consumed during query execution; released when query completes
INSERT / COPY INTO	CUs consumed during data write; compaction may add background CUs
UPDATE / DELETE	Higher CU cost due to read-modify-write nature of Delta operations
Statistics Update	Background operation — consumes CUs proportional to table size
Data Compaction	Background service — managed by Fabric, consumes CUs automatically
Pipeline Data Load	CUs consumed per pipeline activity run

INFO: Use the Fabric Capacity Metrics App in Power BI to monitor CU consumption per workload, user, and operation type. This is the primary tool for capacity planning.

3. Burstable Capacity & Smoothing

Fabric supports bursting — a workload can temporarily consume more CUs than the provisioned capacity. The platform uses a smoothing algorithm to average consumption over a 24-hour rolling window.

How smoothing works:

- If a workload bursts above capacity, it accumulates a 'CU debt'
- The debt is repaid during idle periods when actual consumption is below provisioned capacity
- If debt accumulates too fast or isn't repaid, the workload may be throttled
- Throttling delays query execution — it does not cancel or fail queries
- Critical workloads can be prioritized using workspace-level capacity management settings

WARNING: Sustained bursting without idle recovery periods will lead to throttling. Monitor CU debt in the Capacity Metrics App and scale up capacity before throttling impacts SLAs.

4. Workload Management

Fabric Warehouse uses autonomous workload management — there are no manual resource pools, workload groups, or Resource Governor-style configurations as in SQL Server. The platform automatically manages query concurrency and resource allocation.

Traditional SQL Server	Fabric Warehouse Equivalent
Resource Governor pools	Automatic — managed by Fabric engine
Max degree of parallelism (MAXDOP)	Automatic — engine decides parallelism
Query memory grants	Automatic — engine manages per-query memory
Workload groups	Not available — use capacity SKU sizing instead
Query Store forced plans	Not available — use T-SQL query hints if needed
Priority scheduling	Not available — all queries treated equally

5. The Distributed SQL Engine

The Fabric Warehouse SQL engine is a distributed query processor built for analytical workloads. It translates T-SQL queries into distributed execution plans that read columnar Parquet data from OneLake.

Engine capabilities:

- Vectorized execution: Processes data in batches (vectors) rather than row-by-row for maximum throughput
- Predicate pushdown: Filters are applied at the Parquet file read level, reducing I/O
- Column pruning: Only the columns referenced in the query are read from Parquet files
- Statistics-based optimization: Query optimizer uses column statistics to choose efficient plans
- Automatic caching: Frequently accessed data segments are cached to reduce OneLake reads
- Cross-database queries: T-SQL can JOIN across Warehouses and Lakehouses in the same workspace

6. Query Lifecycle

Understanding the lifecycle of a query helps DBAs diagnose performance issues and interpret DMV data correctly.

Stage	Description	DMV to Monitor
Parsing	T-SQL syntax validation and parse tree generation	sys.dm_exec_requests (status=running)
Binding	Object resolution — tables, columns, types validated	sys.dm_exec_requests
Optimization	Cost-based plan generation using statistics	sys.dm_exec_requests
Compilation	Distributed execution plan created	sys.dm_exec_requests
Execution	Plan executed across distributed nodes	sys.dm_exec_requests (status=running)
Result Return	Results sent to client session	sys.dm_exec_sessions
Completion	Query logged to history	queryinsights.exec_requests_history

7. Supported T-SQL Features

Feature Category	Supported Items
DML Statements	SELECT, INSERT, UPDATE, DELETE, MERGE, COPY INTO
DDL Statements	CREATE/ALTER/DROP TABLE, VIEW, PROCEDURE, FUNCTION, SCHEMA
Query Constructs	CTEs, subqueries, UNION/INTERSECT/EXCEPT, window functions, PIVOT
Joins	INNER, LEFT/RIGHT/FULL OUTER, CROSS, self-joins, cross-database joins
Aggregations	GROUP BY, HAVING, ROLLUP, CUBE, GROUPING SETS
Window Functions	ROW_NUMBER, RANK, DENSE_RANK, LEAD, LAG, SUM/AVG OVER
Data Types	All standard SQL data types including DATETIME2, DECIMAL, NVARCHAR, etc.
Stored Procedures	Full support including parameters, variables, control flow
Views	Standard views and inline TVFs
Transactions	BEGIN/COMMIT/ROLLBACK TRANSACTION — Snapshot Isolation only
Constraints	PRIMARY KEY, UNIQUE, NOT NULL (enforced at write time)
Statistics	AUTO-CREATE and manual CREATE STATISTICS supported

8. Unsupported SQL Features

WARNING: The following SQL Server features are NOT available in Fabric Warehouse. Attempting to use them will result in errors.

Unsupported Feature	Recommended Alternative
Row-level triggers	Implement validation logic in stored procedures or pipelines
Foreign key constraints	Enforce referential integrity in ETL processes
Clustered indexes	Parquet columnar format provides equivalent read efficiency
Non-clustered indexes	Use statistics and partitioning for query optimization
Columnstore indexes	All storage is already columnar (Parquet)
SQL Agent Jobs	Use Fabric Data Factory pipelines or scheduled notebooks
Database Mail	Use Azure Logic Apps or Power Automate for alerts
Linked Servers	Use cross-database T-SQL queries within Fabric workspace
READ COMMITTED isolation	Snapshot Isolation is the only supported level
TRUNCATE TABLE	Use DELETE FROM table; or DROP and recreate

9. Client Connectivity & Tooling

Fabric Warehouse exposes a standard TDS (Tabular Data Stream) endpoint, compatible with any SQL Server client tool.

Tool	Connection Method	Use Case
SSMS (SQL Server Management Studio)	SQL Endpoint URL + Azure AD auth	Query, object browser, scripting
Azure Data Studio	SQL Endpoint URL + Azure AD auth	Query, notebooks, extensions
Fabric Portal Query Editor	Browser-based, auto-connected	Quick queries, schema browsing
Power BI Desktop	DirectLake or DirectQuery via endpoint	Report development
Python (pyodbc/sqlalchemy)	ODBC connection string	Programmatic data access
JDBC clients	JDBC connection string	Java-based applications
DBT	Fabric Warehouse adapter	Data transformation workflows

INFO: Connection string format: Server=.datawarehouse.fabric.microsoft.com; Database=; Authentication=ActiveDirectoryInteractive

10. Concurrency & Session Limits

Fabric Warehouse supports high concurrency but has platform limits that DBAs should be aware of when designing applications and ETL pipelines.

Limit	Value	Notes
Max concurrent queries	Varies by capacity SKU	Higher SKU = more concurrency
Max query execution time	No hard timeout	Long queries may be throttled under capacity pressure
Max result set size	No hard limit	Very large results consume significant CUs
Max table columns	1,024	Exceeding causes DDL error
Max stored procedure nesting	8 levels	Matches SQL Server limit
Max connections	Platform managed	Connections auto-queued when at limit
Session idle timeout	Platform managed	Idle sessions automatically closed